Gate
CS & DA

Artificial Intelligence

Adversarial Search

Lecture No. 1

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

Topic — B$B algo [uninfo.]

Topic — Beam Search.

Topic

Topic

Topic

Turn on Slide map

Topics to be Covered

Topic Adveserial Search

Topic Minimax

Topic $\alpha\beta$ Pruning

Topic

Topic

Adversarial Search

- Adversarial Search: A search in scenarios where multiple agents (players) have conflicting objectives, and each agent's success is often measured against the other's failure. Unlike traditional search problems (like pathfinding), adversarial search considers the presence of an opponent actively working against the agent's goal.

Minimax algo

- Back tracking
- (DFS)

2 players

- Min ⇒ Cost/Result minimize
- Max ⇒ Cost/Result maximize

Both have Conflicting Interest.

Chance is always given to min/max player in alternative manner

finally we have to find the path/Choise that must be taken by Player to get best Result.

Minimax Algorithm:

- The Minimax algorithm is a recursive, decision-making algorithm used in two-player games, particularly zero-sum games where one player's gain is another player's loss. It systematically explores the game tree to determine the optimal move for a player, assuming that the opponent also plays optimally.
- It is a specialized Search Algo that returns optimal Sequence move for a player in a Zero-Sum Game.
- Recursive/ Back tracking algo which is used in decision making and Game theory and uses recursion to Search through Game Tree.
- Algo Computes minimax decision for current state.
- We have 2 players : max and min player. Max player select the maximum value and min player select the minimum value
- Depth-First Search Algo is used for exploration of Complete Game Tree.
- Used in Chess, Tic Tac Toe and other 2 player Games

Minimax Algorithm:

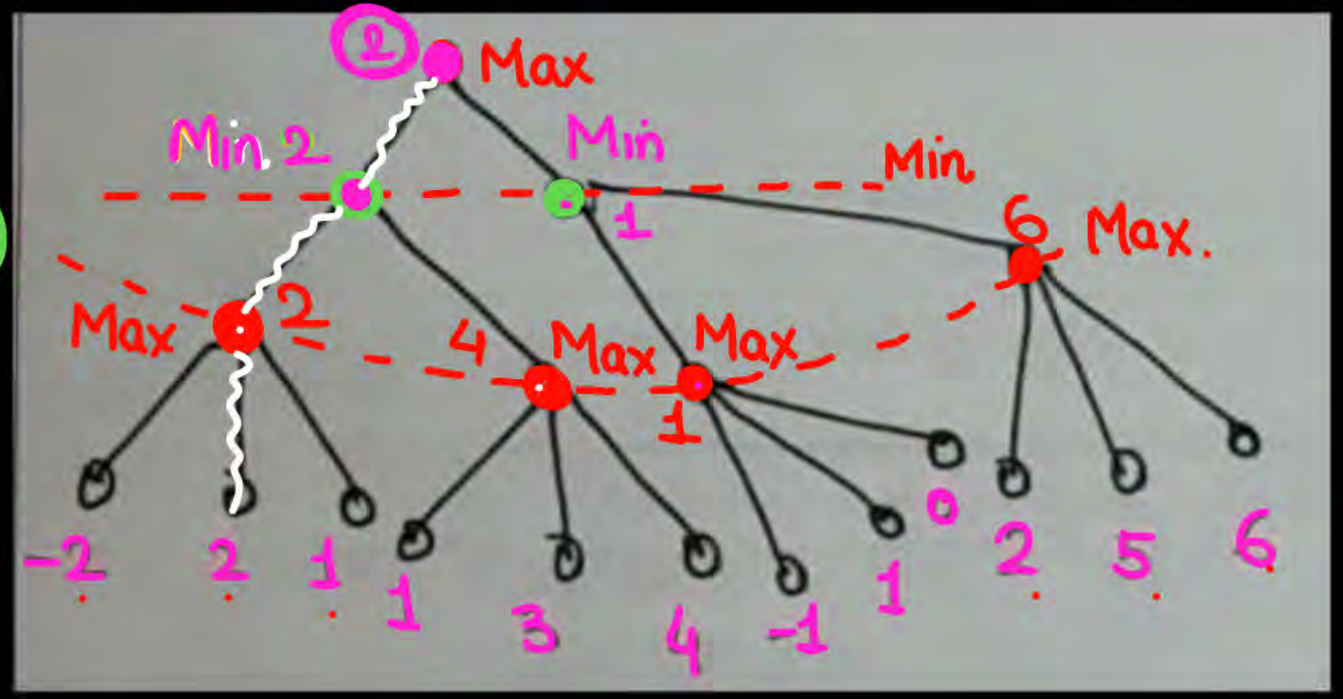
- Initial values for Max and Min player : + infinity and – Infinity
- This is the backtracking algorithm
- Complete and optimal solution
- Time and Space complexity → $O(b^d)$ Both.
- In Minimax algorithm we use DFS

- The best path for Max player:

Gametree (no weight, no heuristic)

- So for max player Best Result = 2

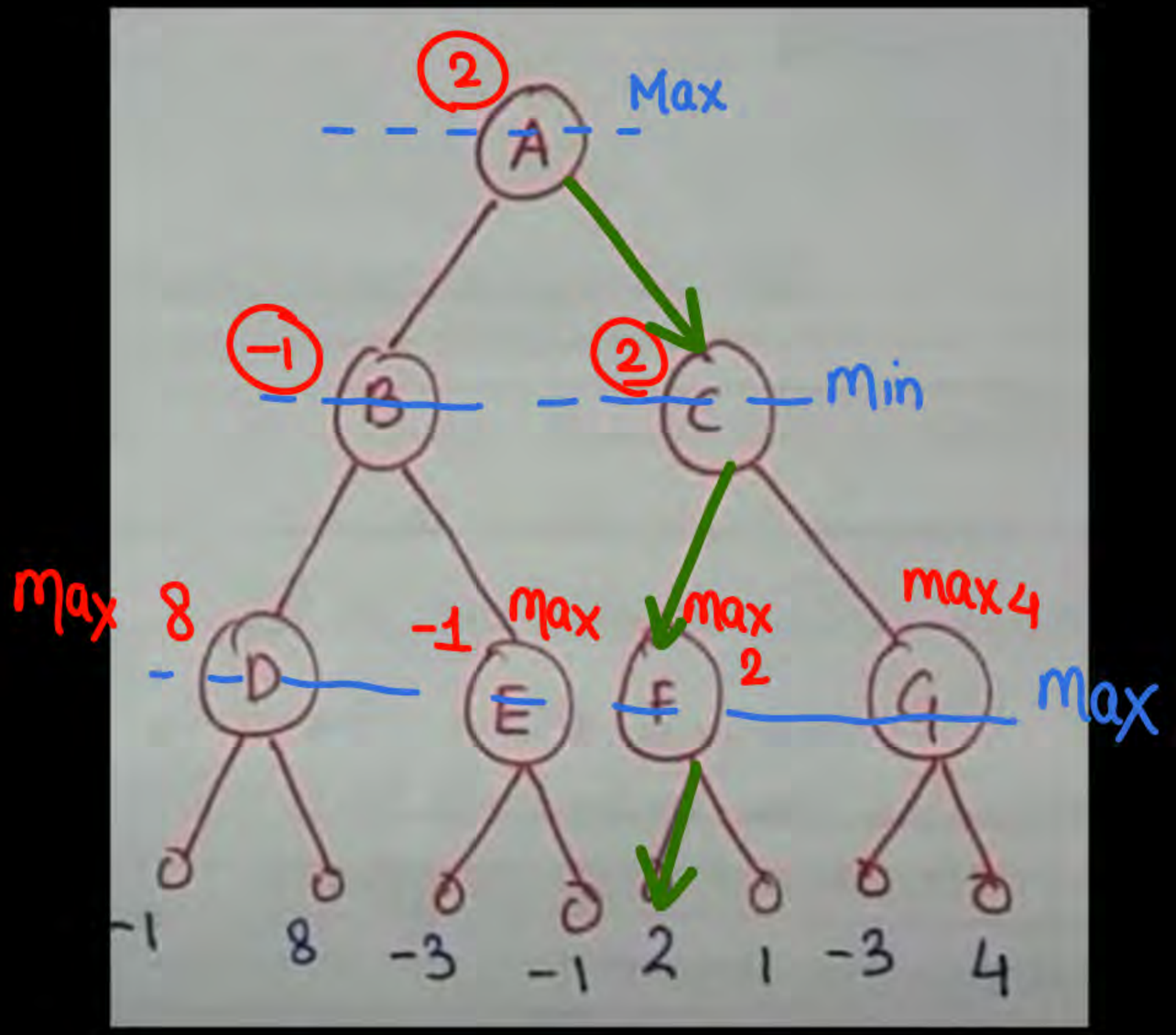
Path is also marked.



- The best path for Max player:

2

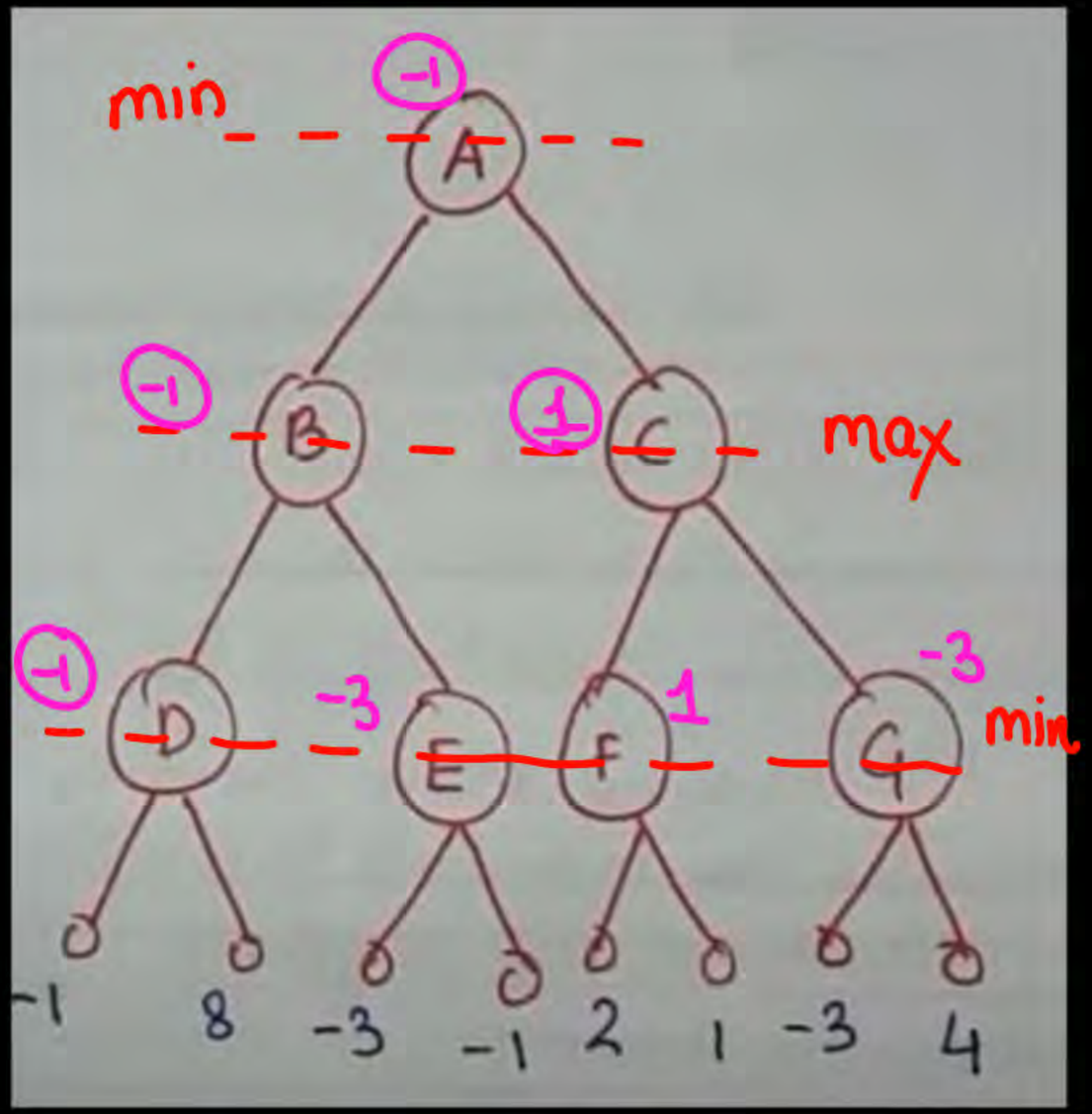
$$\left(\underset{max}{A} \rightarrow \overset{min}{C} \rightarrow F \rightarrow \underset{max}{2}\right)$$



- The best path for ~~Max~~ Min player:

$\rightarrow$ (-1)

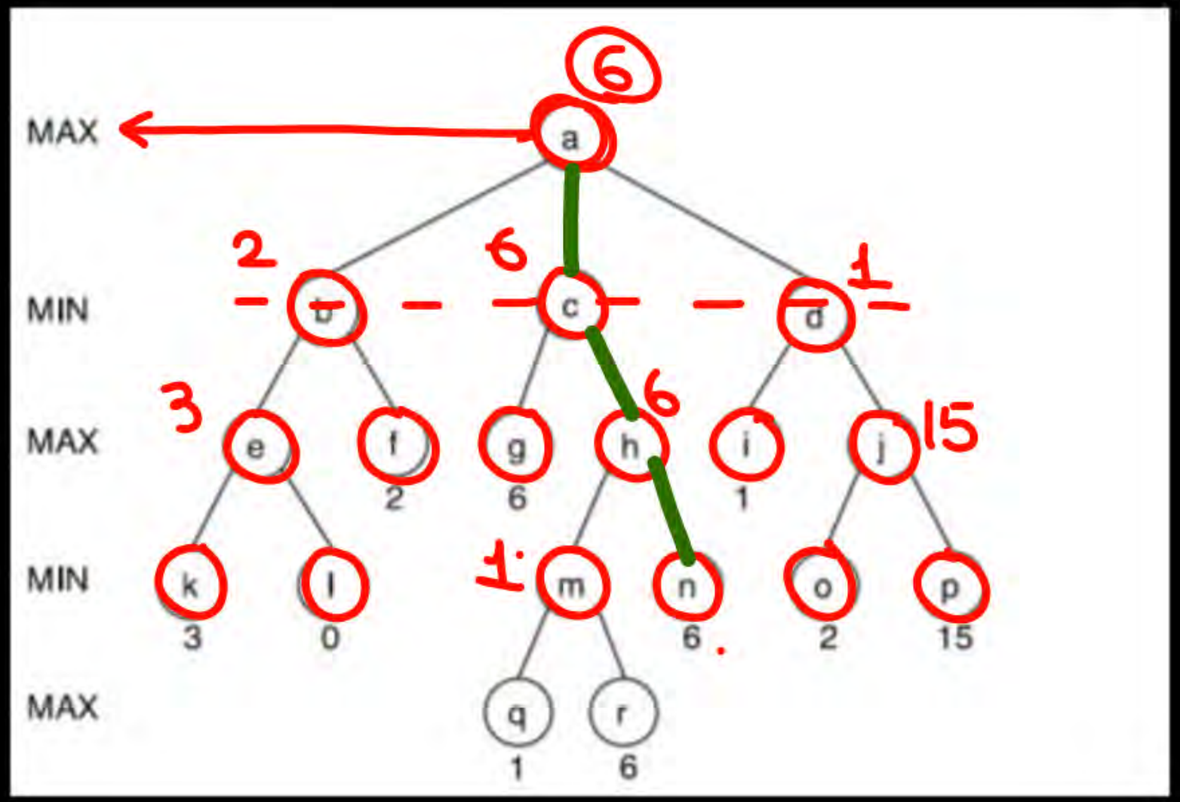
- $A \xrightarrow{\text{min}} B \xrightarrow{\text{max}} D \xrightarrow{\text{min}}$ (-1) •



Q.1 done	Let b be the branching factor of a search tree. If the optimal goal is reached after d actions from the initial state, in the worst case, how many times will the initial state be expanded for iterative deepening depth first search (IDDFS) and iterative Deepening A* search (IDA*)?
(A)	IDDFS - d, IDA* -d.
(B)	IDDFS - d, IDA* -b^d
(C)	IDDFS - b^d, IDA* -d.
(D)	IDDFS - b^d, IDA* -b^d.

- Find the move best for Player A

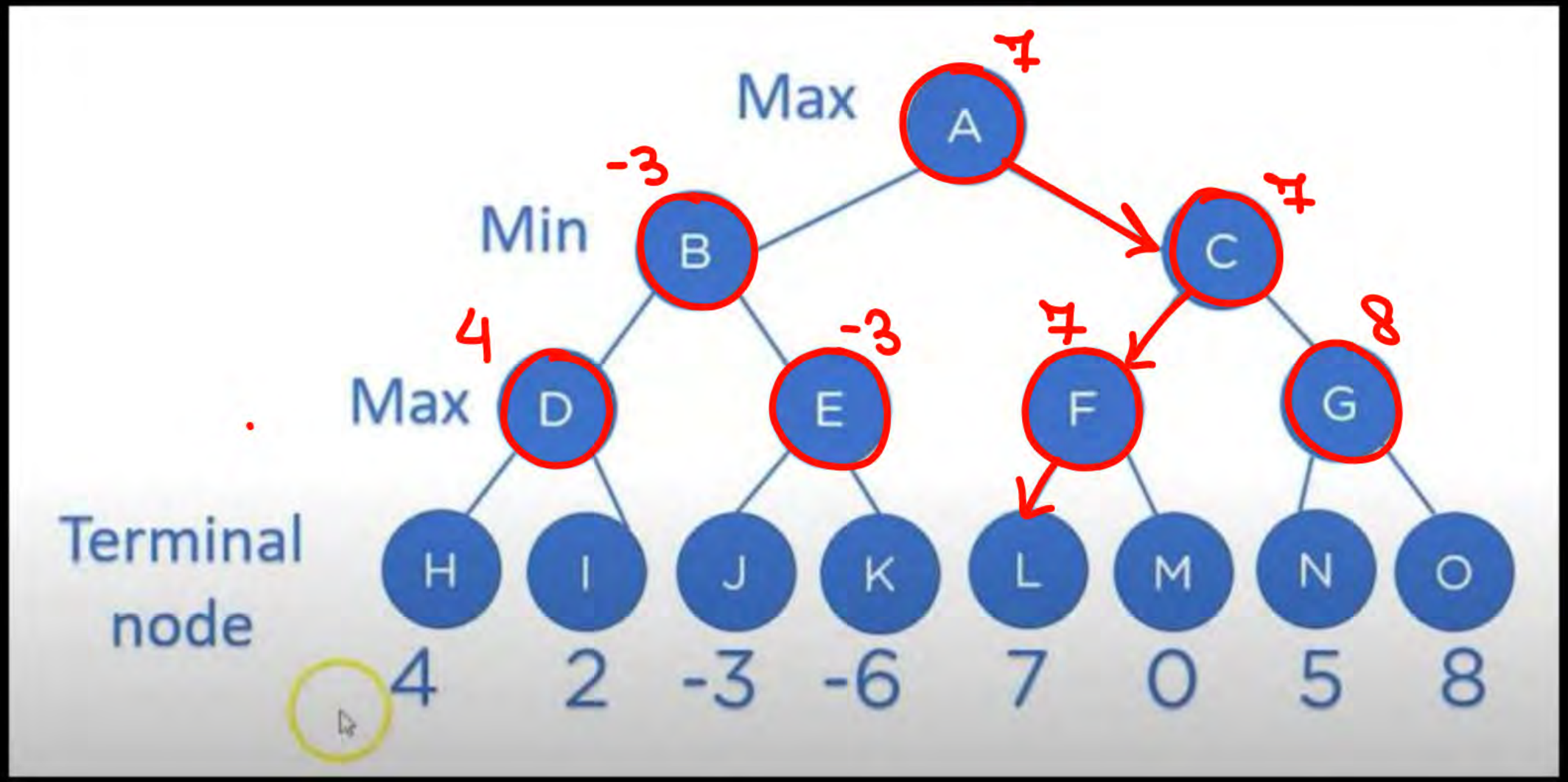
A→C→G ✓
A→C→h
n



- Find the move best for Player A

- Find the move best for Player A

```
          A
        / | \
     B1   B2   B3
    / \  / \  / \
  C1  C2 C3  C4 C5  C6
  |   |  |   |  |   |
  7   4  3   6  8   5
```


Max
A
7
Min
B
-3
C
7
Max
D
4
E
-3
F
7
G
8
Terminal node
H
I
J
K
L
M
N
O
4
2
-3
-6
7
0
5
8

Minimax Algorithm:

- Conceptual Overview Two Players:
  - Often referred to as the "Maximizer" and the "Minimizer.
  - "Maximizer: Tries to maximize their score.
  - Minimizer: Tries to minimize the Maximizer's score (which is equivalent to maximizing their own score if you flip the sign).
- Game Tree: A tree structure representing all possible moves (edges) from a given game state (nodes).
- Root Node: Represents the current state of the game.
- Leaf Nodes: Represent terminal states where the game ends (e.g., win, loss, or draw).

Minimax Algorithm:

- Minimax Principle
- The algorithm assumes that both players play optimally.
- It simulates all possible moves from the current state down to terminal states, then back-propagates the values up the tree to make the best decision at the root.

Minimax Algorithm:

- Minimax Algorithm Steps
- Generate the Game Tree: Start from the root node (current game state) and recursively expand all possible moves down to a certain depth or until terminal nodes (end of the game) are reached.
- Evaluate Terminal Nodes:
  - At the terminal nodes (leaf nodes), assign values based on the outcome:
  - Positive value (e.g., +1): A win for the Maximizer.
  - Negative value (e.g., -1): A win for the Minimizer.
  - Zero (0): A draw.

Minimax Algorithm:

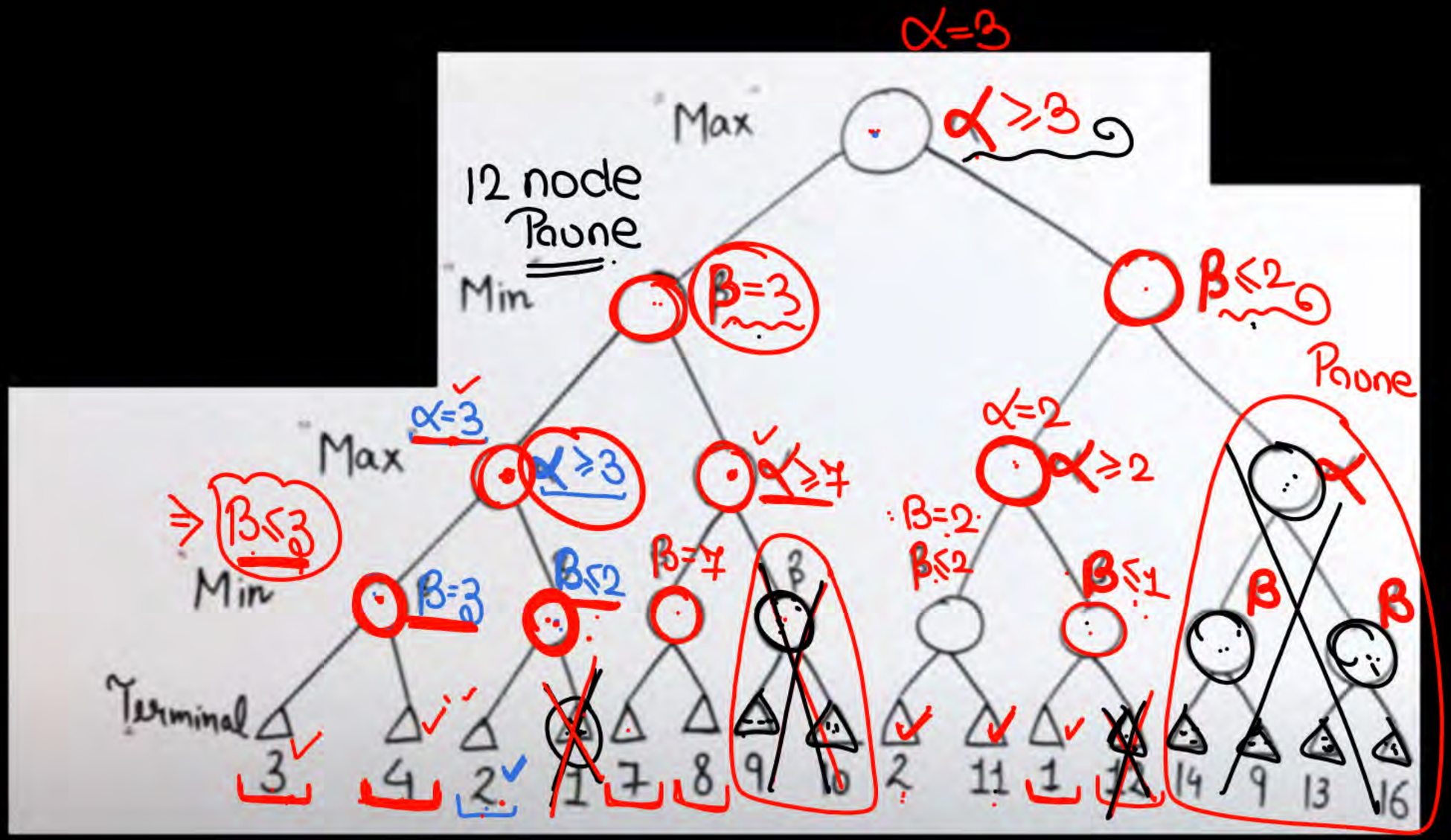
- Minimax Algorithm Steps
- Backpropagate the Values:
  - Backpropagate the values from the terminal nodes up to the root.
  - At Maximizer nodes (Max nodes), select the maximum value from the child nodes.
  - At Minimizer nodes (Min nodes), select the minimum value from the child nodes.
  - Select the Best Move: At the root node (current game state), the Maximizer chooses the move corresponding to the child node with the highest value.

Alpha-Beta Pruning

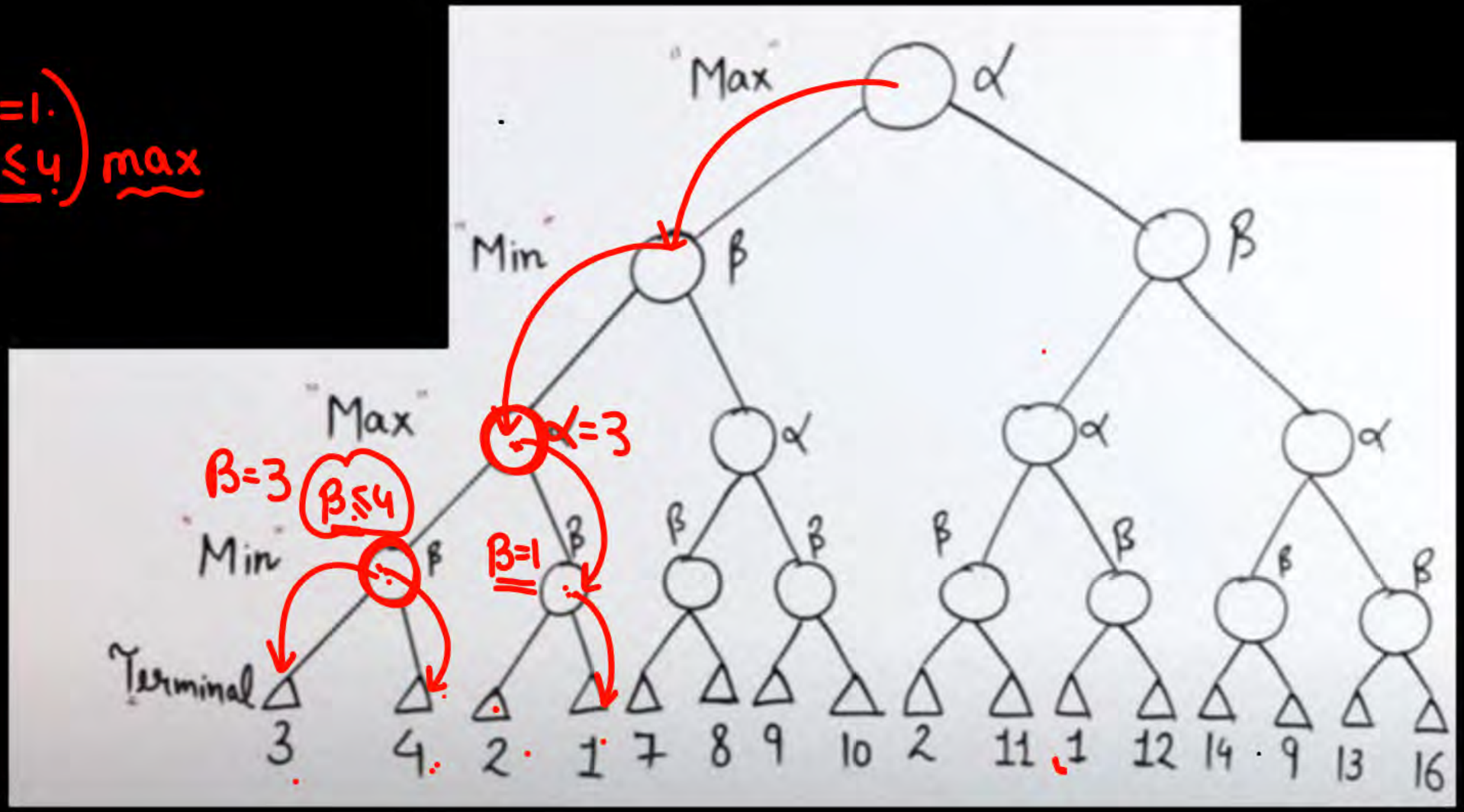
- Alpha-beta pruning is an optimization technique for the minimax algorithm, commonly used in decision-making for two-player games like chess, tic-tac-toe, and other adversarial games. The main purpose of alpha-beta pruning is to reduce the number of nodes evaluated by the minimax algorithm, thereby speeding up the decision-making process without affecting the final decision.

Alpha-Beta Pruning

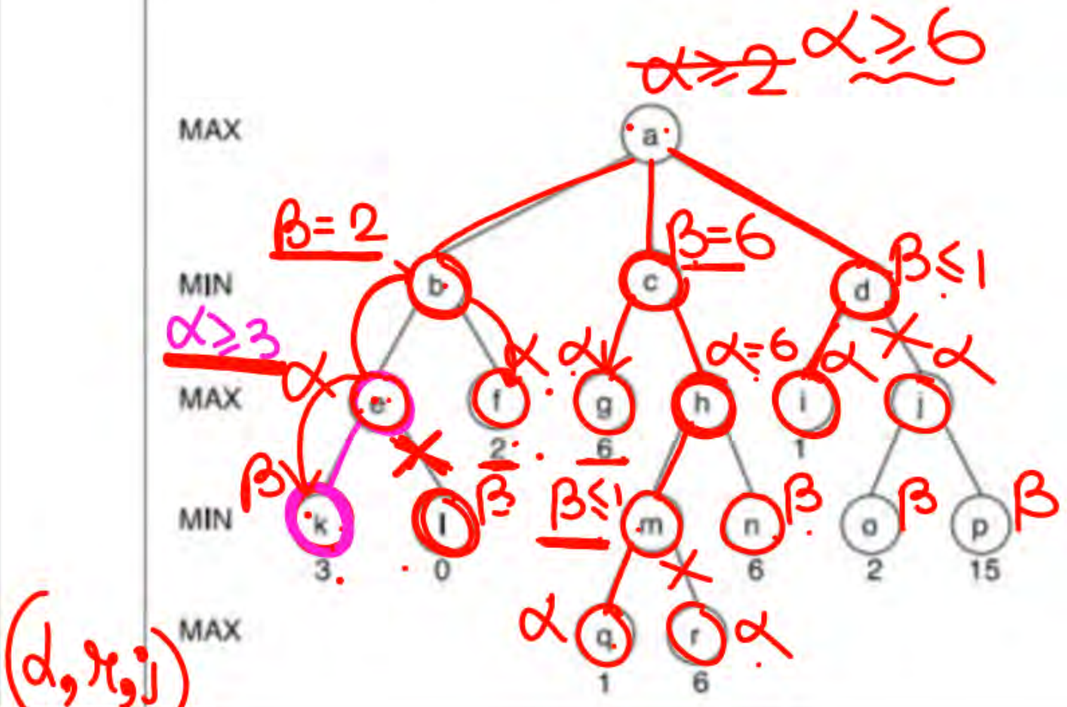
- This is advanced version of minimax algorithm
- Because minimax algorithm has to explore all the nodes thus time complexity was _____
- But in this new algorithm we explore less number of nodes
- Alpha : is for max player
- Beta : is for min player


α=3
Max
α≥3
12 node
Prune
Min
β=3
β≤2
Prune
Max
α=3
α≥3
α≥7
α=2
α≥2
⇒ β≤3
Min
β=3
β≤2
β=7
β
β=2
β≤2
β≤1
β
β
Terminal
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

$x=1$
$y \leq 4$ max


Max
α
Min
β
β
Max
α=3
α
α
α
β=3
β≤4
Min
β
β=1
β
β
β
β
β
β
β
β
Terminal
3
4
2
1
7
8
9
10
2
11
1
12
14
9
13
16

Q.31 Consider the game tree shown below. The value below each node is the output of the utility function. The subtrees rooted at which of these nodes will be pruned because of alpha-beta pruning?



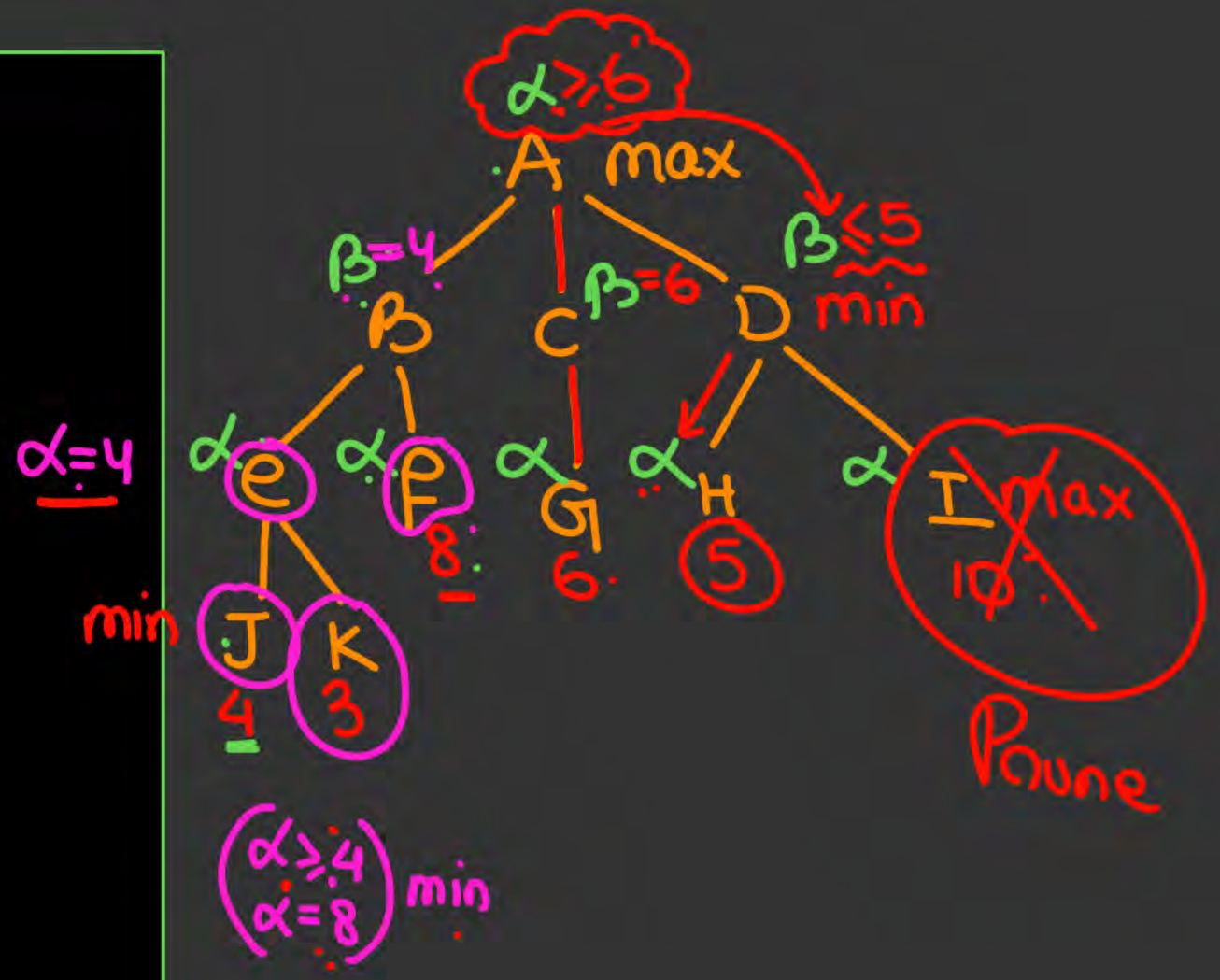
(d, r, j)

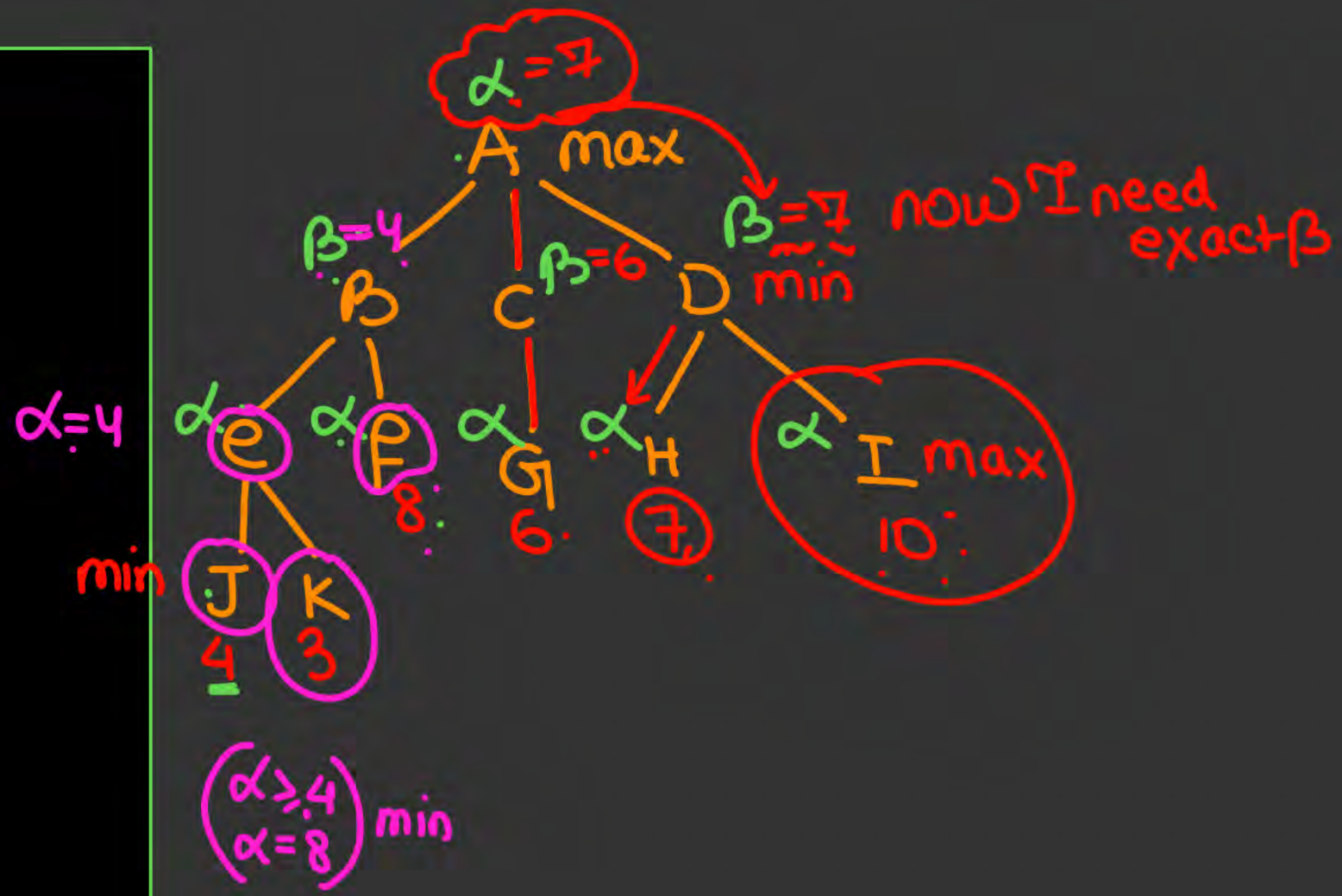
(A) m and j

(B) r and j

(C) h and p

(D) no nodes are pruned


α≥6
A
max
β=4
β=6
β≤5
min
B
C
D
α=4
α
E
α
F
α
G
α
H
α
I
max
10
8
6
5
min
J
K
4
3
Prune
(α≥4
α=8)
min


α=7
A max
β=4
β=6
β=7 min
now I need exact β
B
C
D
α=4
α
E
α
F
8
α
G
6
α
H
7
α
I max
10
min
J
4
K
3
(α≥4
α=8) min

Alpha-Beta Pruning

- Alpha-beta pruning is an optimization technique for the minimax algorithm, commonly used in decision-making for two-player games like chess, tic-tac-toe, and other adversarial games. The main purpose of alpha-beta pruning is to reduce the number of nodes evaluated by the minimax algorithm, thereby speeding up the decision-making process without affecting the final decision.

THANK - YOU